

1 Activation Function Implementations:

Implementation of activations.Linear:

```
1 class Linear(Activation):
2     def __init__(self):
3         super().__init__()
4
5     def forward(self, Z: np.ndarray) -> np.ndarray:
6         """Forward pass for f(z) = z.
7
8         Parameters
9         -----
10        Z input pre-activations (any shape)
11
12        Returns
13        -----
14        f(z) as described above applied elementwise to `Z`
15        """
16        return Z
17
18    def backward(self, Z: np.ndarray, dY: np.ndarray) -> np.ndarray:
19        """Backward pass for f(z) = z.
20
21        Parameters
22        -----
23        Z input to `forward` method
24        dY gradient of loss w.r.t. the output of this layer
25           same shape as `Z`
26
27        Returns
28        -----
29        gradient of loss w.r.t. input of this layer
30        """
31        return dY
```

Implementation of activations.Sigmoid:

```
1 class Sigmoid(Activation):
2     def __init__(self):
3         super().__init__()
4
5     def forward(self, Z: np.ndarray) -> np.ndarray:
6         """Forward pass for sigmoid function:
7         f(z) = 1 / (1 + exp(-z))
8
9         Parameters
10        -----
11        Z input pre-activations (any shape)
12
13        Returns
14        -----
15        f(z) as described above applied elementwise to `Z`
16        """
17        ### YOUR CODE HERE ###
18        sigmoid = 1 / (1 + np.exp(-Z))
19        return sigmoid
20
21    def backward(self, Z: np.ndarray, dY: np.ndarray) -> np.ndarray:
22        """Backward pass for sigmoid.
23
24        Parameters
25        -----
26        Z input to `forward` method
27        dY gradient of loss w.r.t. the output of this layer
28           same shape as `Z`
29
```

```

30     Returns
31     -----
32     gradient of loss w.r.t. input of this layer
33     """
34     ### YOUR CODE HERE ###
35     # the derivative of sigmoid function is sigmoid(z) * (1 - sigmoid(z))
36     sigmoid = self.forward(Z)
37     dsigmoid_dZ = sigmoid * (1 - sigmoid)
38     dZ = dY * dsigmoid_dZ # "*" represents elementwise multiplication
39     return dZ

```

Implementation of activations.ReLU:

```

1  class ReLU(Activation):
2      def __init__(self):
3          super().__init__()
4
5      def forward(self, Z: np.ndarray) -> np.ndarray:
6          """Forward pass for relu activation:
7              f(z) = z if z >= 0
8                  0 otherwise
9
10             Parameters
11             -----
12             Z  input pre-activations (any shape)
13
14             Returns
15             -----
16             f(z) as described above applied elementwise to `Z`
17             """
18             ### YOUR CODE HERE ###
19             return np.maximum(0, Z)
20
21     def backward(self, Z: np.ndarray, dY: np.ndarray) -> np.ndarray:
22         """Backward pass for relu activation.
23
24         Parameters
25         -----
26         Z  input to `forward` method
27         dY  gradient of loss w.r.t. the output of this layer
28             same shape as `Z`
29
30         Returns
31         -----
32         gradient of loss w.r.t. input of this layer
33         """
34         ### YOUR CODE HERE ###
35         reLU = self.forward(Z)
36         dYdZ = (reLU > 0).astype(float)
37         dZ = dY * dYdZ # "*" represents elementwise multiplication
38                       # rather than matrix multiplication
39         return dZ

```

Implementation of activations.SoftMax:

```

1  class SoftMax(Activation):
2      def __init__(self):
3          super().__init__()
4
5      def forward(self, Z: np.ndarray) -> np.ndarray:
6          """Forward pass for softmax activation.
7              Hint: The naive implementation might not be numerically stable.
8
9             Parameters
10             -----
11             Z  input pre-activations (any shape)
12

```

```

13     Returns
14     -----
15     f(z) as described above applied elementwise to `Z`
16     """
17     ### YOUR CODE HERE ###
18     max_Z = np.max(Z, axis=-1, keepdims=True)
19     exp_Z = np.exp(Z - max_Z)
20     exp_Z_sum = np.sum(exp_Z, axis=-1, keepdims=True)
21     sigma = exp_Z / exp_Z_sum
22     return sigma
23
24 def backward(self, Z: np.ndarray, dY: np.ndarray) -> np.ndarray:
25     """Backward pass for softmax activation.
26
27     Parameters
28     -----
29     Z    input to `forward` method
30     dY    gradient of loss w.r.t. the output of this layer
31           same shape as `Z`
32
33     Returns
34     -----
35     gradient of loss w.r.t. input of this layer
36     """
37     ### YOUR CODE HERE ###
38     sigma = self.forward(Z)
39     batch_size, n_classes = Z.shape
40
41     dZ = np.zeros_like(Z)
42     # if i == j, dsigma_i / dZ_j = sigma_i * (1 - sigma_i)
43     # if i != j, dsigma_i / dZ_j = -sigma_i * sigma_j
44     # which means dsigma / dZ can be rewritten as (diag(sigma) - sigma * sigma^T)
45     for i in range(batch_size):
46         sigma_example = sigma[i]
47         diag = np.diagflat(sigma_example)
48         outer_product = np.outer(sigma_example, sigma_example) # calculate sigma * sigma^T
49         dsigma_dZ_example = diag - outer_product
50         dZ[i] = dY[i] @ dsigma_dZ_example # for each example in the mini-batch,
51                                           # calculate dL/dZ = dL/dsigma * dsigma/dZ
52
53     return dZ

```

2 Layer Implementations:

Implementation of layers.FullyConnected:

```

1 class FullyConnected(Layer):
2     """A fully-connected layer multiplies its input by a weight matrix, adds
3     a bias, and then applies an activation function.
4     """
5
6     def __init__(
7         self, n_out: int, activation: str, weight_init="xavier_uniform"
8     ) -> None:
9
10         super().__init__()
11         self.n_in = None
12         self.n_out = n_out
13         self.activation = initialize_activation(activation)
14
15         # instantiate the weight initializer
16         self.init_weights = initialize_weights(weight_init, activation=activation)
17
18     def _init_parameters(self, X_shape: Tuple[int, int]) -> None:
19         """Initialize all layer parameters (weights, biases)."""

```

```

20     self.n_in = X_shape[1]
21
22     ### BEGIN YOUR CODE ###
23
24     W = self.init_weights((self.n_in, self.n_out))
25     b = np.zeros((1, self.n_out))
26
27     self.parameters = OrderedDict({"W": W, "b": b}) # DO NOT CHANGE THE KEYS
28     self.cache: OrderedDict = OrderedDict({"Z": [], "X": []}) # cache for backprop
29     self.gradients: OrderedDict = OrderedDict(
30         {"W": np.zeros_like(W), "b": np.zeros_like(b)}
31     ) # parameter gradients initialized to zero
32         # MUST HAVE THE SAME KEYS AS `self.parameters`
33
34     ### END YOUR CODE ###
35
36 def forward(self, X: np.ndarray) -> np.ndarray:
37     """Forward pass: multiply by a weight matrix, add a bias, apply activation.
38     Also, store all necessary intermediate results in the `cache` dictionary
39     to be able to compute the backward pass.
40
41     Parameters
42     -----
43     X input matrix of shape (batch_size, input_dim)
44
45     Returns
46     -----
47     a matrix of shape (batch_size, output_dim)
48     """
49     # initialize layer parameters if they have not been initialized
50     if self.n_in is None:
51         self._init_parameters(X.shape)
52
53     ### BEGIN YOUR CODE ###
54
55     # perform an affine transformation and activation
56     Z = X @ self.parameters["W"] + self.parameters["b"]
57     out = self.activation.forward(Z)
58
59     # store information necessary for backprop in `self.cache`
60     self.cache["X"] = X
61     self.cache["Z"] = Z
62
63     ### END YOUR CODE ###
64
65     return out
66
67 def backward(self, dLdY: np.ndarray) -> np.ndarray:
68     """Backward pass for fully connected layer.
69     Compute the gradients of the loss with respect to:
70     1. the weights of this layer (mutate the `gradients` dictionary)
71     2. the bias of this layer (mutate the `gradients` dictionary)
72     3. the input of this layer (return this)
73
74     Parameters
75     -----
76     dLdY gradient of the loss with respect to the output of this layer
77         shape (batch_size, output_dim)
78
79     Returns
80     -----
81     gradient of the loss with respect to the input of this layer
82     shape (batch_size, input_dim)
83     """
84     ### BEGIN YOUR CODE ###
85

```

```

86     # unpack the cache
87     Z, X = self.cache["Z"], self.cache["X"]
88
89     # compute the gradients of the loss w.r.t. all parameters as well as the
90     # input of the layer
91     dZ = self.activation.backward(Z, dLdY)
92     dX = dZ @ self.parameters["W"].T
93     dW = X.T @ dZ
94     # In section 4.2.1, we calculated dLdb = 1^T @ dLdz,
95     # where 1 is a column of ones, shape of (m, 1), and m is the batch size.
96     # Here, 1^T @ dLdz = (1, m) @ (m, n_out) = (1, n_out),
97     # which is equivalent to summing dLdz over axis 0.
98     db = np.sum(dZ, axis=0, keepdims=True)
99
100    # store the gradients in `self.gradients`
101    # the gradient for self.parameters["W"] should be stored in
102    # self.gradients["W"], etc.
103    self.gradients["W"] = dW
104    self.gradients["b"] = db
105
106    ### END YOUR CODE ###
107
108    return dX

```

Implementation of layers.Pool2D:

```

1  class Pool2D(Layer):
2      """Pooling layer, implements max and average pooling."""
3
4      def __init__(
5          self,
6          kernel_shape: Tuple[int, int],
7          mode: str = "max",
8          stride: int = 1,
9          pad: Union[int, Literal["same"], Literal["valid"]] = 0,
10     ) -> None:
11
12         if type(kernel_shape) == int:
13             kernel_shape = (kernel_shape, kernel_shape)
14
15         self.kernel_shape = kernel_shape
16         self.stride = stride
17
18         if pad == "same":
19             self.pad = ((kernel_shape[0] - 1) // 2, (kernel_shape[1] - 1) // 2)
20         elif pad == "valid":
21             self.pad = (0, 0)
22         elif isinstance(pad, int):
23             self.pad = (pad, pad)
24         else:
25             raise ValueError("Invalid Pad mode found in self.pad.")
26
27         self.mode = mode
28
29         if mode == "max":
30             self.pool_fn = np.max
31             self.arg_pool_fn = np.argmax
32         elif mode == "average":
33             self.pool_fn = np.mean
34
35         self.cache = {
36             "out_rows": [],
37             "out_cols": [],
38             "X_pad": [],
39             "p": [],
40             "pool_shape": [],
41         }

```

```

42     self.parameters = {}
43     self.gradients = {}
44
45 def forward(self, X: np.ndarray) -> np.ndarray:
46     """Forward pass: use the pooling function to aggregate local information
47     in the input. This layer typically reduces the spatial dimensionality of
48     the input while keeping the number of feature maps the same.
49
50     As with all other layers, please make sure to cache the appropriate
51     information for the backward pass.
52
53     Parameters
54     -----
55     X input array of shape (batch_size, in_rows, in_cols, channels)
56
57     Returns
58     -----
59     pooled array of shape (batch_size, out_rows, out_cols, channels)
60     """
61     ### BEGIN YOUR CODE ###
62
63     # implement the forward pass
64     kernel_height, kernel_width = self.kernel_shape
65     n_examples, in_rows, in_cols, channels = X.shape
66     out_rows = (in_rows + 2 * self.pad[0] - kernel_height) // self.stride + 1
67     out_cols = (in_cols + 2 * self.pad[1] - kernel_width) // self.stride + 1
68     # X shape: (n_examples, in_rows, in_cols, in_channels)
69     # we need to pad the 2nd and 3rd dimensions (height and width)
70     # and keep the rest the same which is done by set to (0,0) for those dimensions.
71     X_pad = np.pad(X, ((0, 0), (self.pad[0], self.pad[0]), (self.pad[1], self.pad[1]), (0, 0)),
72                    ↪ mode='constant')
73     X_pool = np.zeros((n_examples, out_rows, out_cols, channels))
74     for i in range(out_rows):
75         for j in range(out_cols):
76             X_pool_slice = X_pad[:,
77                               i * self.stride : i * self.stride + kernel_height,
78                               j * self.stride : j * self.stride + kernel_width,
79                               :] # the effective receptive field for the pooling w.r.t. output pixel (i,
80                               ↪ j). shape of (n_examples, kernel_height, kernel_width, in_channels)
81             if self.mode == "max":
82                 X_pool[:, i, j, :] = np.max(X_pool_slice, axis=(1, 2))
83             elif self.mode == "average":
84                 X_pool[:, i, j, :] = np.mean(X_pool_slice, axis=(1, 2))
85
86     # cache any values required for backprop
87     self.cache["out_rows"] = out_rows
88     self.cache["out_cols"] = out_cols
89     self.cache["X_pad"] = X_pad
90     self.cache["p"] = self.pool_fn
91     self.cache["pool_shape"] = self.kernel_shape
92
93     ### END YOUR CODE ###
94
95     return X_pool
96
97 def backward(self, dLdY: np.ndarray) -> np.ndarray:
98     """Backward pass for pooling layer.
99
100     Parameters
101     -----
102     dLdY gradient of loss with respect to the output of this layer
103     shape (batch_size, out_rows, out_cols, channels)
104
105     Returns
106     -----
107     gradient of loss with respect to the input of this layer

```

```

106     shape (batch_size, in_rows, in_cols, channels)
107     """
108     ### BEGIN YOUR CODE ###
109
110     # perform a backward pass
111     # unpack the cache
112     out_rows, out_cols = self.cache["out_rows"], self.cache["out_cols"]
113     X_pad = self.cache["X_pad"]
114     pool_fn, pool_shape = self.cache["p"], self.cache["pool_shape"]
115
116     n_examples, padded_in_rows, padded_in_cols, channels = X_pad.shape
117     kernel_height, kernel_width = pool_shape
118
119     dX_pad = np.zeros_like(X_pad)
120     for i in range(out_rows):
121         for j in range(out_cols):
122             X_pool_slice = X_pad[:,
123                               i * self.stride : i * self.stride + kernel_height,
124                               j * self.stride : j * self.stride + kernel_width,
125                               :]
126             if self.mode == "max":
127                 # create a mask of the same shape as X_pool_slice
128                 # where the max value indices are set to 1, rest are 0
129                 max_mask = (X_pool_slice == np.max(X_pool_slice, axis=(1, 2), keepdims=True))
130                 # for max pooling, gradient flows only to the neurons which achieved the max
131                 # all other neurons get zero gradient
132                 # which is equivalent to elementwise multiplying dLdY with the mask
133                 dX_pad[:,
134                       i * self.stride : i * self.stride + kernel_height,
135                       j * self.stride : j * self.stride + kernel_width,
136                       :] \
137                 += np.einsum("nc,nijc->nijc", dLdY[:, i, j, :], max_mask, optimize=True)
138             elif self.mode == "average":
139                 # for average pooling, gradient is equally divided among all neurons
140                 # create a mask of the same shape as X_pool_slice
141                 # where all values are 1 / (kernel_height * kernel_width)
142                 avg_mask = np.ones_like(X_pool_slice) / (kernel_height * kernel_width)
143                 dX_pad[:,
144                       i * self.stride : i * self.stride + kernel_height,
145                       j * self.stride : j * self.stride + kernel_width,
146                       :] \
147                 += np.einsum("nc,nijc->nijc", dLdY[:, i, j, :], avg_mask, optimize=True)
148
149     # unpad dX_pad to get dX
150     dX = dX_pad[:,
151                 self.pad[0] : padded_in_rows - self.pad[0],
152                 self.pad[1] : padded_in_cols - self.pad[1],
153                 :]
154
155     ### END YOUR CODE ###
156
157     return dX

```

Implementation of layers.Conv2D.__init__:

```

1  def __init__(
2      self,
3      n_out: int,
4      kernel_shape: Tuple[int, int],
5      activation: str,
6      stride: int = 1,
7      pad: str = "same",
8      weight_init: str = "xavier_uniform",
9  ) -> None:
10
11      super().__init__()
12      self.n_in = None

```

```

13     self.n_out = n_out
14     self.kernel_shape = kernel_shape
15     self.stride = stride
16     self.pad = pad
17
18     self.activation = initialize_activation(activation)
19     self.init_weights = initialize_weights(weight_init, activation=activation)

```

Implementation of layers.Conv2D._init_parameters:

```

1  def _init_parameters(self, X_shape: Tuple[int, int, int, int]) -> None:
2      """Initialize all layer parameters and determine padding."""
3      self.n_in = X_shape[3]
4
5      W_shape = self.kernel_shape + (self.n_in,) + (self.n_out,)
6      W = self.init_weights(W_shape)
7      b = np.zeros((1, self.n_out))
8
9      self.parameters = OrderedDict({"W": W, "b": b}) # DO NOT CHANGE THE KEYS
10     self.cache = OrderedDict({"Z": [], "X": []}) # cache for backprop
11     self.gradients = OrderedDict({"W": np.zeros_like(W), "b": np.zeros_like(b)}) # parameter
12                                     # MUST HAVE THE SAME KEYS AS
13                                     # → `self.parameters`
14
15     if self.pad == "same":
16         self.pad = ((W_shape[0] - 1) // 2, (W_shape[1] - 1) // 2)
17     elif self.pad == "valid":
18         self.pad = (0, 0)
19     elif isinstance(self.pad, int):
20         self.pad = (self.pad, self.pad)
21     else:
22         raise ValueError("Invalid Pad mode found in self.pad.")

```

Implementation of layers.Conv2D.forward:

```

1  def forward(self, X: np.ndarray) -> np.ndarray:
2      """Forward pass for convolutional layer. This layer convolves the input
3      `X` with a filter of weights, adds a bias term, and applies an activation
4      function to compute the output. This layer also supports padding and
5      integer strides. Intermediates necessary for the backward pass are stored
6      in the cache.
7
8      Parameters
9      -----
10     X input with shape (batch_size, in_rows, in_cols, in_channels)
11
12     Returns
13     -----
14     output feature maps with shape (batch_size, out_rows, out_cols, out_channels)
15     """
16     if self.n_in is None:
17         self._init_parameters(X.shape)
18
19     W = self.parameters["W"]
20     b = self.parameters["b"]
21
22     kernel_height, kernel_width, in_channels, out_channels = W.shape
23     n_examples, in_rows, in_cols, in_channels = X.shape
24     kernel_shape = (kernel_height, kernel_width)
25
26     ### BEGIN YOUR CODE ###
27
28     # implement a convolutional forward pass
29     # compute output dimensions
30     out_rows = (in_rows + 2 * self.pad[0] - kernel_height) // self.stride + 1
31     out_cols = (in_cols + 2 * self.pad[1] - kernel_width) // self.stride + 1

```

```

32     Z = np.zeros((n_examples, out_rows, out_cols, out_channels))
33     # first pad the input X
34     # X shape: (n_examples, in_rows, in_cols, in_channels)
35     # we need to pad the 2nd and 3rd dimensions (height and width)
36     # and keep the rest the same which is done by set to (0,0) for those dimensions.
37     X_pad = np.pad(X, ((0, 0), (self.pad[0], self.pad[0]), (self.pad[1], self.pad[1]), (0, 0)),
38                     ↪ mode='constant')
39     # use np.einsum for the convolution operation
40     # Z[d1, d2, n] = (X conv W)[d1, d2, n]
41     # = sum_i sum_k sum_c W[i, k, c, n] * X[d1 + i, d2 + k, c] + b[n]
42     for i in range(out_rows):
43         for j in range(out_cols):
44             X_conv = X_pad[:,
45                             i * self.stride : i * self.stride + kernel_height,
46                             j * self.stride : j * self.stride + kernel_width,
47                             :] # the effective receptive field for the convolution w.r.t. output pixel
48                             ↪ (i,j). shape of (n_examples, kernel_height, kernel_width,
49                             ↪ in_channels)
50             # W.shape: kernel_height, kernel_width, in_channels, out_channels
51             # X.shape: n_examples, in_rows, in_cols, in_channels
52             # out.shape: n_examples, out_rows, out_cols, out_channels
53             # einsum:
54             # sum(kernel_height * in_rows)
55             # sum(kernel_width * in_cols)
56             # keep n_examples, out_channels
57             Z[:, i, j, :] = np.einsum("ijkl,nijk->nl", W, X_conv, optimize=True) + b
58
59     out = self.activation.forward(Z)
60
61     # cache any values required for backprop
62     self.cache["Z"] = Z
63     self.cache["X"] = X
64
65     ### END YOUR CODE ###
66
67     return out

```

Implementation of layers.Conv2D.backward:

```

1  def backward(self, dLdY: np.ndarray) -> np.ndarray:
2      """Backward pass for conv layer. Computes the gradients of the output
3      with respect to the input feature maps as well as the filter weights and
4      biases.
5
6      Parameters
7      -----
8      dLdY gradient of loss with respect to output of this layer
9          shape (batch_size, out_rows, out_cols, out_channels)
10
11      Returns
12      -----
13      gradient of the loss with respect to the input of this layer
14      shape (batch_size, in_rows, in_cols, in_channels)
15      """
16      ### BEGIN YOUR CODE ###
17
18      # perform a backward pass
19
20      # unpack the cache
21      Z, X = self.cache["Z"], self.cache["X"]
22      W, b = self.parameters["W"], self.parameters["b"]
23
24      # compute the gradients of the loss w.r.t. all parameters as well as the
25      # input of the layer
26      dZ = self.activation.backward(Z, dLdY)
27      kernel_height, kernel_width, in_channels, out_channels = W.shape
28      n_examples, in_rows, in_cols, in_channels = X.shape

```

```

29     _, out_rows, out_cols, out_channels = Z.shape # also dZ.shape. Here _ is n_examples
30     # pad X and dX, same as in forward pass
31     X_pad = np.pad(X, ((0, 0), (self.pad[0], self.pad[0]), (self.pad[1], self.pad[1]), (0, 0)),
32                     ↪ mode='constant')
33     dX_pad = np.zeros_like(X_pad)
34     dW = np.zeros_like(W)
35     db = np.zeros_like(b)
36     for i in range(out_rows):
37         for j in range(out_cols):
38             X_conv = X_pad[:,
39                             i * self.stride : i * self.stride + kernel_height,
40                             j * self.stride : j * self.stride + kernel_width,
41                             :] # same as in forward pass
42             # compute dW, db, dX
43             # dW[i, k, c, f] = sum_{d1, d2} dZ[d1, d2, f] * X[d1 + i, d2 + k, c]
44             dW += np.einsum("nf,nikc->ikcf", dZ[:, i, j, :], X_conv, optimize=True)
45             # db[f] = sum_{d1, d2} dZ[d1, d2, f]
46             db += np.einsum("nf->f", dZ[:, i, j, :], optimize=True).reshape(1, -1)
47             # dX[x, y, c] += sum_{d1, d2, f} dZ[d1, d2, f] * W[x - d1, y - d2, c, f]
48             dX_pad[:,
49                     i * self.stride : i * self.stride + kernel_height,
50                     j * self.stride : j * self.stride + kernel_width,
51                     :] \
52             += np.einsum("nf,ikcf->nikc", dZ[:, i, j, :], W, optimize=True)
53             # unpad dX_pad to get dX
54             dX = dX_pad[:,
55                         self.pad[0] : in_rows + self.pad[0],
56                         self.pad[1] : in_cols + self.pad[1],
57                         :]
58
59     # store the gradients in `self.gradients`
60     # the gradient for self.parameters["W"] should be stored in
61     # self.gradients["W"], etc.
62     self.gradients["W"] = dW
63     self.gradients["b"] = db
64
65     ### END YOUR CODE ###
66
67     return dX

```

3 Loss Function Implementations:

Implementation of losses.CrossEntropy:

```

1  class CrossEntropy(Loss):
2      """Cross entropy loss function."""
3
4      def __init__(self, name: str) -> None:
5          self.name = name
6
7      def __call__(self, Y: np.ndarray, Y_hat: np.ndarray) -> float:
8          return self.forward(Y, Y_hat)
9
10     def forward(self, Y: np.ndarray, Y_hat: np.ndarray) -> float:
11         """Computes the loss for predictions `Y_hat` given one-hot encoded labels
12         `Y`.
13
14         Parameters
15         -----
16         Y        one-hot encoded labels of shape (batch_size, num_classes)
17         Y_hat     model predictions in range (0, 1) of shape (batch_size, num_classes)
18
19         Returns
20         -----
21         a single float representing the loss

```

```

22     """
23     ### YOUR CODE HERE ###
24     # L = - 1 / m * \sum_i^m {y_i @ ln(y_hat_i)}
25     m = Y.shape[0]
26     loss = - (1 / m) * np.sum(Y * np.log(Y_hat + np.finfo(float).eps)) # elementwise multiplication
27     # y_i @ ln(y_hat_i) = sum over classes of y_ij * ln(y_hat_ij)
28     # then sum over all examples i,
29     # which is equivalent to `np.sum(Y * np.log(Y_hat))`
30     # `np.finfo(float).eps` is added to avoid log(0)
31     return loss
32
33 def backward(self, Y: np.ndarray, Y_hat: np.ndarray) -> np.ndarray:
34     """Backward pass of cross-entropy loss.
35     NOTE: This is correct ONLY when the loss function is SoftMax.
36
37     Parameters
38     -----
39     Y        one-hot encoded labels of shape (batch_size, num_classes)
40     Y_hat    model predictions in range (0, 1) of shape (batch_size, num_classes)
41
42     Returns
43     -----
44     the gradient of the cross-entropy loss with respect to the vector of
45     predictions, `Y_hat`
46     """
47     ### YOUR CODE HERE ###
48     m = Y.shape[0]
49     dY_hat = - (1 / m) * (Y / Y_hat) # Here Y / Y_hat is elementwise division
50     return dY_hat

```

4 Model Implementations:

Implementation of `models.NeuralNetwork.forward`:

```

1  def forward(self, X: np.ndarray) -> np.ndarray:
2      """One forward pass through all the layers of the neural network.
3
4      Parameters
5      -----
6      X    design matrix whose must match the input shape required by the
7           first layer
8
9      Returns
10     -----
11     forward pass output, matches the shape of the output of the last layer
12     """
13     ### YOUR CODE HERE ###
14     # Iterate through the network's layers.
15     # Input layer's input is X.
16     Y = X
17     for layer in self.layers:
18         Y = layer.forward(Y)
19     return Y

```

Implementation of `models.NeuralNetwork.backward`:

```

1  def backward(self, target: np.ndarray, out: np.ndarray) -> float:
2      """One backward pass through all the layers of the neural network.
3      During this phase we calculate the gradients of the loss with respect to
4      each of the parameters of the entire neural network. Most of the heavy
5      lifting is done by the `backward` methods of the layers, so this method
6      should be relatively simple. Also make sure to compute the loss in this
7      method and NOT in `self.forward`.
8
9      Note: Both input arrays have the same shape.

```

```

10
11     Parameters
12     -----
13     target the targets we are trying to fit to (e.g., training labels)
14     out     the predictions of the model on training data
15
16     Returns
17     -----
18     the loss of the model given the training inputs and targets
19     """
20     ### YOUR CODE HERE ###
21     # Compute the loss.
22     # Backpropagate through the network's layers.
23     # For output layer, the loss gradient is computed with target labels and the final output
24     #   ↪ labels.
25     loss = self.loss.forward(target, out)
26     dY = self.loss.backward(target, out)
27     for layer in reversed(self.layers): # backward pass, so reverse order
28         dY = layer.backward(dY)
29     return loss

```

Implementation of `models.NeuralNetwork.predict`:

```

1  def predict(self, X: np.ndarray, Y: np.ndarray) -> Tuple[np.ndarray, float]:
2      """Make a forward and backward pass to calculate the predictions and
3      loss of the neural network on the given data.
4
5      Parameters
6      -----
7      X input features
8      Y targets (same length as `X`)
9
10     Returns
11     -----
12     a tuple of the prediction and loss
13     """
14     ### YOUR CODE HERE ###
15     # Do a forward pass. Maybe use a function you already wrote?
16     # Get the loss. Remember that the `backward` function returns the loss.
17     Y_hat = self.forward(X)
18     loss = self.loss.forward(Y, Y_hat)
19     return (Y_hat, loss)

```